CB Guideline: Making a Task Hard for Opus 5.5 (Hardness v2)

This guide replaces the hardness advice we built on Opus 4.8. It is based on 46 real Enterprise Atlas tasks where Opus 5.5 scored under 75% on the baseline, which comes to 270 runs and 2,843 graded rubric rows. For every row Opus failed, we checked the judge's reasoning against the author's evidence to find out what actually beat the model. Keep the one-page checklist (Appendix) open while you design.

Part 1. How hardness is scored

- **What gets measured:** six Opus 5.5 baseline rollouts with no hints. Each run gets a weighted rubric score (weight-5 rows count five times as much as weight-1 rows), and the six scores are averaged.
- **The limit depends on your difficulty mode:**
    - **Easy:** under 90%.
    - **Medium:** under 85%. A task with no mode is treated as Medium.
    - **Hard:** under 70%.
- **What this means:** to pass, your weight-5 rows have to depend on things Opus gets wrong. A task where Opus nails every heavy row and misses a few light ones will not get under the limit.
- **The pre-check is a single run.** Treat it as an early warning, not a verdict.

Part 2. What changed from v1

- **Corrections still work, but only when they are out of the way.** v1 said Opus checks stale-vs-newer messages by default. That's still true when the newer message sits in the same channel or thread. A correction in a different place still beats Opus 5.5: a Slack message that withdraws a GitHub approval, a Confluence comment that reopens a Linear ticket, a calendar note that says the work is already done. This showed up in 28 of the 46 tasks.
- **"Proving done" is now the top trap.** The biggest single cause of lost score was Opus accepting one source's "done" (a merged PR, a closed ticket) when another service showed a required part was missing.
- **Length doesn't make a task harder.** The 14 Normal tasks averaged 66% and the 32 Long-Horizon tasks averaged 67%. The lowest-scoring tasks had about the same number of rows and tool calls as the 70–75% tasks. What set them apart was the kind of decision they asked for.

- **Plain volume and plain arithmetic are now easy.** Opus 5.5 reliably makes long inventories, does dozens of routine writes and adds up totals correctly. It fails when the inputs to a total are wrong, not when it has to add.

Part 3. What beats Opus 5.5 (do these)

The traps are listed in order of how much score they cost Opus across the 46 tasks.

1. Proving "done" across services

- **How it works:** one source says an item is finished. Another service shows a required piece never landed.
- **What we saw:** in one task, all six runs marked tickets as on production because their PRs were deployed. The table imports those tickets needed were only on dev, which the inline PR reviews and the deployment log showed. In another, all six runs called VFX work ready even though the season-pass animation was still missing.
- **How to build it:** make the "done" source believable and easy to find. Put the missing piece in a different service, where an agent only finds it by checking each part. Make sure the data defines what "on prod", "merged" or "signed off" means.

2. A correction somewhere else

- **How it works:** the agent finds a convincing answer, and a later record in another place overrides it.
- **What we saw:** in one task, Oliver withdrew his approval of a PR in Slack. All six runs still counted the GitHub approval, so the unapproved totals came out one too low (45 and 109 instead of 46 and 110). In another, a later Confluence QA comment reopened two tickets that Linear showed as signed off and fixed. All six runs marked them fixed.
- **How to build it:** put a dated correction in a thread reply, a page comment, a review or another service, and link it clearly to the original decision (same item, same person). The correction must change a number, a status or a write.

3. Follow-through that a rule requires

- **How it works:** the agent makes the right calls but skips a write that a rule in the data requires.
- **What we saw:** one task's market keep/cut calls passed 6/6, but a comment from Leonard said every moved ticket needs a comment with its market and call. All six runs skipped those comments on every ticket.
- **How to build it:** put the follow-through rule in a real handoff, team rule or earlier promise. Make the destination easy to find.
- **Be careful:** grade a few of these writes, not dozens. Thirty identical comment rows are one trap, not thirty, and a rubric built like that is fragile.

4. Dates, windows and dependencies

- **How it works:** a cutoff, timezone or prerequisite changes which item qualifies or when work can happen.
- **What we saw:** one task missed an item that closed at 00:41 Pacific on May 1, inside the cutoff, because it belonged to an older batch. Another had six runs that found free days but scheduled the wrong work on them, because prerequisites and a moved deadline weren't applied.
- **How to build it:** state the governing timezone and method in a natural plan or message. Include one boundary case or one changed leave record that affects a real assignment.

5. A rule or exception hidden in the data

- **How it works:** a definition or exception in older records changes an answer that otherwise looks obvious.
- **What we saw:** in the lowest-scoring task (31%), Opus did the bulk Drive cleanup perfectly but missed every exception. An old email said to keep board-deck copies, another kept device lists, and a canceled-meeting note said to trash the January Netflix deck. All 22 failing rows came from those exceptions.
- **How to build it:** give the agent a natural reason to look for the policy (the request mentions the cleanup, the budget or the roll-up). Put the deciding exception in a record the agent can find from there.

6. Different names for the same thing

- **How it works:** a person, company or item is named differently in different places.
- **What we saw:** one task's investor update named only the contact (Gustaf), not the firm (Sony). All six runs missed Sony's pass. In another, asset names differed from existing ticket titles, so runs created duplicates instead of reusing the tickets.
- **How to build it:** use a real alias, a person-to-company link or a differently titled existing ticket. Put the mapping in the contacts, an earlier conversation or the ticket history. Don't use spelling tricks.

7. Items outside the obvious filter

- **How it works:** a few in-scope items don't show up under the obvious search.
- **What we saw:** one task's request list had 48 items. Two carried a Bug label instead of the usual one, and all six runs dropped them, so every email listed 46. In another, branches without PRs in other repos were never listed.
- **How to build it:** put two or three genuinely in-scope items just outside the obvious filter, with a rule in the data that brings them in. Grade each one on its own row.

8. Wrong inputs to a total

- **How it works:** the math is easy. The hard part is which numbers go into it.
- **What we saw:** in one task, all six runs missed a second software seat that was added in an old thread, and three missed a refund. Basic totals in the same task passed 6/6.

- **How to build it:** put adjustments, refunds, revised claims or rounding rules in normal billing records or messages. Grade the corrected input and the final figure on separate rows.

9. Work someone already did

- **How it works:** an item looks like it still needs action, but a record elsewhere shows it's done.
- **What we saw:** a canceled calendar invite carried a note saying the first two charges were already recorded, and a DM said January–March were done. All six runs created records for them anyway and labeled 20 emails instead of 17.
- **How to build it:** put a believable completion note in a different place from the original request. Grade that the agent leaves those items alone.

Part 4. What no longer works on its own (avoid these)

These passed 6/6 even in tasks that scored low overall:

- **Long inventories and routine bulk writes.** The 31% Drive task still listed exactly 44 files and did every ordinary move, rename, share and trash correctly.
- **Straight totals.** Adding up charges or forecasts passes when the inputs are clean.
- **Clear calls from visible evidence.** Seven market keep/cut calls with their CPI and D1 figures all passed.
- **Pulling facts from one spec page and publishing them.**
- **A caveat Opus can see.** "Not signed off" and "the test didn't measure the creative" both passed when the evidence was in plain view.
- **Adding steps, services or rows.** None of these lowered scores by themselves.

Part 5. Design for hardness before you write

Add these to your decision map (see "Building a Task That Passes", Part 3):

1. **At least two deciding facts that need checking across sources.** Two separate decisions, not one answer repeated in many places.
2. **For each one, name the convincing wrong source and the record that beats it.** Who wrote each, when and with what authority, so the winning source is clear.
3. **The miss must change a weight-5 row.** A status, number, inclusion or write, not a comment.
4. **The link must be findable.** Every alias, invoice ID or build number must map back to the item through records the persona can see.
5. **Rules live in the records.** The prompt points to a natural starting place; the exception, definition or update sits in the data.

Part 6. Rubric rows that make hardness count

- **Put weight 5 on the trap rows.** If the trap only touches weight-1 rows, the score won't move.
- **One condition per row.** Inclusion, amount, status, recipient and content each get their own row.
- **Grade the end state, not the route.** Check the corrected value or status, not whether the agent searched a certain way.
- **Grade the write itself.** A final "done" message doesn't replace the email, ticket update or sheet row. An item left unchanged can also be the right outcome, so add a guard row for it.
- **Grade meaning, not wording.** Exact strings only for IDs and formats the data requires. Allow a sensible tolerance on figures.

Fake difficulty to avoid. These make a task score low for the wrong reason, and reviewers will catch them:

- a row whose evidence adds a condition its title doesn't state (title says "includes every PR", evidence adds "and no others")
- one row that bundles many clauses of an email or template
- rows that fail an answer for missing a literal phrase when the fact is there
- dozens of rows grading the same downstream effect of one miss
- injected data that never reached the environment. Check it through the agent's tools, not just in your SQL.

Part 7. Self-check before you run /hardness

Ask the portal assistant each of these:

6. **Done check:** "Which source says [item] is done, and does any other service show a part of it missing?" If nothing is missing anywhere, the done-check trap isn't there.
7. **Same-place check:** "Is the correction to [fact] in the same channel or thread as the original?" If yes, Opus will likely catch it. Move it.
8. **Filter check:** "What do I get from the obvious search for [set]? Which in-scope items does it miss?" If it misses nothing, the list is just volume.
9. **Input check:** "Which records change the inputs to [total]?" If none, the math is routine.
10. **Decision flip:** "If an agent missed [the hard record], which weight-5 rows would fail?" If none, raise the weight or move the evidence onto the decision path.

Part 8. If the run comes back too easy

- **Look at which heavy rows passed 6/6.** Those are the parts Opus finds easy. Don't add more of them.

- **Move a correction out of the obvious place** into a thread reply, a comment, a review or another service.
- **Add a "done" source with a missing piece**, or a completion note somewhere else.
- **Move one or two in-scope items outside the obvious filter.**
- **Put the deciding fact behind an alias** that a contact or history record resolves.
- **Don't** add rows, items or steps, and don't spell the rule out in the prompt. One task went from failing to 100% after its rules were written into the prompt.

Part 9. Worked examples (real tasks)

1. A withdrawn approval

Title: A correction in Slack overrides GitHub

The trap: The task asks for a memo on how many PRs merged without approval. GitHub shows Oliver approved PR #338. In Slack, he later withdrew that approval.

What Opus 5.5 did: All six runs counted #338 as approved. The memo said 45 and 109 unapproved PRs instead of 46 and 110, and the sheet marked #338 as approved by Oliver. All three weight-5 rows failed 6/6. The 88 and 178 merged-PR counts passed 6/6.

Why it works (Part 3, trap 2): The correction is in another service, tied to one item, and it changes two graded numbers. Opus counted everything else correctly, so the miss is real and bounded.

2. Deployed, but not on prod

Title: A merged PR isn't proof that every part shipped

The trap: A server sign-off register asks whether each ticket is on production. The PRs were deployed, but the table imports for three tickets were only on dev, which the inline PR reviews and the deployment log showed.

What Opus 5.5 did: All six runs marked all three tickets as on prod, and all three weight-5 rows failed 6/6.

Why it works (Part 3, trap 1): The "done" signal is real and easy to find. The missing piece is in a different place and only shows up if the agent checks every part.

3. Already recorded

Title: A completion note in an unexpected place

The trap: The agent has to create a vendor charge record for each Metabase receipt without one. A canceled calendar invite carries a note saying the first two charges were already recorded, and a DM says January–March were done too.

What Opus 5.5 did: All six runs created 20 records instead of 17 and labeled 20 emails instead of 17. Both weight-5 rows failed 6/6.

Why it works (Part 3, trap 9): The note is believable and findable, but it isn't where the request lives, so extra action becomes the error.

4. Exceptions to a bulk cleanup

Title: The rule is easy; the exceptions decide the score

The trap: A Drive cleanup has clear bulk rules in #executives. Older emails and a canceled-meeting note carve out specific files: keep board-deck copies, keep device lists, trash the January Netflix deck.

What Opus 5.5 did: It did the bulk work perfectly. The 44-file list and the ordinary moves, renames and shares all passed 6/6. It missed every exception, and the task scored 31%.

Why it works (Part 3, trap 5): The cleanup request gives the agent a reason to read the rules, but the exceptions sit in older mail it has to connect back to specific files.

5. Two items outside the filter

Title: Put a few in-scope items where the obvious search misses them

The trap: A make-good list should cover 48 requests. Two of them carry a Bug label instead of the usual one.

What Opus 5.5 did: All six runs listed 46, so the weight-5 rows for the full list and for those two requests failed 6/6.

Why it works (Part 3, trap 7): The list is long, but that isn't what makes it hard. Two items need a second look, and they decide the rows.

6. Right calls, missing follow-through

Title: A rule in the data requires a write on every item

The trap: A comment from Leonard on the queue ticket says every moved ticket needs a comment with its market and call.

What Opus 5.5 did: All seven market calls passed 6/6, but no run left the comments.

Why it works, with a caution (Part 3, trap 3): The rule is natural and the miss is real. But this task graded 37 near-identical comment rows, which makes one miss look like 37. Grade a few representative tickets plus one count row instead.

Appendix. One-page checklist

While designing

- At least two separate decisions need checking across sources.
- For each one, name the convincing wrong source and the record that beats it.
- Missing the hard record changes a weight-5 row (status, number, inclusion or write).
- Rules and exceptions live in the records, not the prompt.
- Every alias or ID maps back to its item through records the persona can see.

Pick at least two of these traps

- A "done" source with a missing piece in another service.
- A correction in a different place from the original.
- A follow-through rule that requires a write.
- A date, timezone or dependency that changes which item qualifies.
- An exception in older records that overrides the obvious rule.
- A person, company or item named differently in different places.
- Two or three in-scope items outside the obvious filter.
- An adjustment that changes the inputs to a total.
- A completion note somewhere unexpected.

Don't rely on

- Long lists, many writes, many services or many steps.
- Plain arithmetic on clean inputs.
- A correction in the same channel or thread.
- Rules spelled out in the prompt.

Before /hardness

- Done check, same-place check, filter check, input check and decision flip all pass (Part 7).
- Trap rows are weight 5, one condition each.
- Injected records are visible through the agent's tools.

If it comes back too easy

- Find the heavy rows that passed 6/6 and stop adding more like them.
- Move a correction, add a missing piece or move items outside the filter.
- Don't add rows or steps, and don't write the rules into the prompt.